FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CCS007L
(COMPUTER PROGRAMMING 2)

EXERCISE

6

LINKED LIST

Student Name / Group Name:	Jed L. Hizon	
Members (if Group):	**Name**	**Role**

Section:	AN11 - Term 2
Professor:	Mr. Cezar

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements. [PO: C]

II. COURSE LEARNING OUTCOME/S (CLO)ADDRESSED BY THE LABORATORY EXERCISE

- Apply the fundamental principles of handling CString values, pointers and memory allocation, and structures using C++in solving computing activities [CLO: 2]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE At the end of this exercise, students must be able to:

- Perform linked list operation
- Iterate and process each data element from a given linked list.
- Have a full understanding on variation of Linked List
- Create some applied program using linked list
- Create a record management program using doubly linked list

IV. BACKGROUND INFORMATION

A linked list is a sequence of data structures, which are connected together via links.

Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list is the second most-used data structure after array. Following are the important terms to understand the concept of Linked List.

- Link − Each link of a linked list can store a data called an element.
- Next − Each link of a linked list contains a link to the next link called Next.
- LinkedList − A Linked List contains the connection link to the first link called First.

Linked List Representation

Linked list can be visualized as a chain of nodes, where every node points to the next node.



As per the above illustration, following are the important points to be considered.

- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.

Types of Linked List

Following are the various types of linked list.

- Simple Linked List − Item navigation is forward only.
- Doubly Linked List − Items can be navigated forward and backward.
- Circular Linked List − Last item contains link of the first element as next and the first element has a link to the last element as previous.

Basic Operations

Following are the basic operations supported by a list.

- Insertion − Adds an element at the beginning of the list.
- Deletion − Deletes an element at the beginning of the list.
- Display − Displays the complete list.
- Search − Searches an element using the given key.
- Delete − Deletes an element using the given key.

V. GRADING SYSTEM / RUBRIC

Trait	(Excellent)	(Good)	(Fair)	(Poor)

Requirement Specification(30pts)	Able to identify correctly all input and output and provide alternative. **(28-20pts)**	Able to identify correctly all input and output **(25-17pts)**	Able to identify only one input or output **(22-14pts)**	Unable to identify any input and output **(20-11pts)**
Data type(20pts)	Able to apply required data type or data structure and produce correct results **(1820pts)**	Able to apply required data type or data structure and produce partially correct results **(15-17pts)**	Able to identify required data type or data structure but does apply correctly **(12-14pts)**	Unable to identify required data type **(9-11pts)**
Input Validation(20pts)	The program works and meets all specifications. Does exception al checking for errors and out-of- range data **(18-20pts)**	The program works and meets all specifications. Does some checking for errors and out of range data **(15-17pts)**	The program produces correct results but does not display correctly Does not check for errors and out of range data **(12-14pts)**	The program produce s incorrect results **(9-11pts)**
Free from syntax, logic, and runtime errors (10pts)	Unable to run program **(10pts)**	Able to run program but have logic error **(8-9pts)**	Able to run program correctly without any logic error and display inappropriate output **(6-7pts)**	Able to run program correctly without any logic error and display appropriate output **(5pts)**
Delivery (10pts)	The program was delivered on time **(10pts)**	The program was delivered after 5 minutes from the time required. **(89pts)**	The program was delivered after 10 minutes from the time required. **(67pts)**	The program was delivered after 15 (or more) minutes from the time required. **(5pts)**
Use of Comments (10pts)	Specific purpose is noted for each function, control structure, input requirements, and output results. **(10pts)**	Specific purpose is noted for each function and control structure. **(8-9pts)**	Purpose is noted for each function. **(67pts)**	No comments included. **(5pts)**

VI. LABORATORY ACTIVITY

Creating a Player Records Management Program with Linked List

In this guided exercise, you will create a C++ program that manages player records using a linked list.

Follow these steps as a guide to complete the program:

Step 1: Set Up the Linked List

1. Create a new C++ source code file named <lastname>_E6_linked_list.cpp.
2. Include the necessary header files:

```
#include <iostream>
#include <string>
using namespace std;
```

3. Define the Player structure to store player information:

```
struct Player {
   string nickname;
   int age;
   double score1;
   double score2;
};
```

Step 2: Define the Node Structure

1. Define the Node structure to create nodes for the linked list:

```
struct Node {
   Player player;
   Node* next;
};
```

Step 3: Implement the Linked List

1. Declare a pointer to the head of the linked list:

```
Node* head = nullptr;
```

2. Implement a function to add a player record to the linked list:

```
void addRecord(const Player& player) {
   Node* newNode = new Node;
   newNode->player = player;
   newNode->next = head;
   head = newNode;
}
```

3. Implement a function to display all player records in the linked list:

```
void viewRecords() {
   Node* current = head;
   while (current != nullptr) {
      cout << "Nickname: " << current->player.nickname << endl;
      cout << "Age: " << current->player.age << endl;
      cout << "Score 1: " << current->player.score1 << endl;
      cout << "Score 2: " << current->player.score2 << endl;
      cout << "----------------------------------" << endl;
      current = current->next;
   }
}
```

4. Implement a function to compute and display the average score for each player:

```
void computeAverage() {
   Node* current = head;
   while (current != nullptr) {
      double average = (current->player.score1 + current->player.score2) / 2.0;
      cout << "Nickname: " << current->player.nickname << endl;
      cout << "Average Score: " << average << endl;
      cout << "----------------------------------" << endl;
      current = current->next;
   }
}
```

5. Implement functions to find and display players with the highest and lowest average scores:

```
void showMaxAverage() {
    Node* current = head;
    double maxAverage = -1; // Initialize to a non-valid value
    while (current != nullptr) {
        double average = (current->player.score1 + current->player.score2) / 2.0;
        if (average > maxAverage) {
            maxAverage = average;
        }
        current = current->next;
    }
    // Now, iterate again to display players with the max average
    current = head;
    while (current != nullptr) {
        double average = (current->player.score1 + current->player.score2) / 2.0;
        if (average == maxAverage) {
            cout << "Nickname: " << current->player.nickname << endl;
            cout << "Average Score: " << average << endl;
            cout << "---------------------------------" << endl;
        }
        current = current->next;
    }
}
```

```
void showMinAverage() {
    Node* current = head;
    double minAverage = 999999; // Initialize to a large value
    while (current != nullptr) {
        double average = (current->player.score1 + current->player.score2) / 2.0;
        if (average < minAverage) {
            minAverage = average;
        }
        current = current->next;
    }
    // Now, iterate again to display players with the min average
    current = head;
    while (current != nullptr) {
        double average = (current->player.score1 + current->player.score2) / 2.0;
        if (average == minAverage) {
            cout << "Nickname: " << current->player.nickname << endl;
            cout << "Average Score: " << average << endl;
            cout << "---------------------------------" << endl;
        }
        current = current->next;
    }
}
```

Step 4: Implement the Main Function

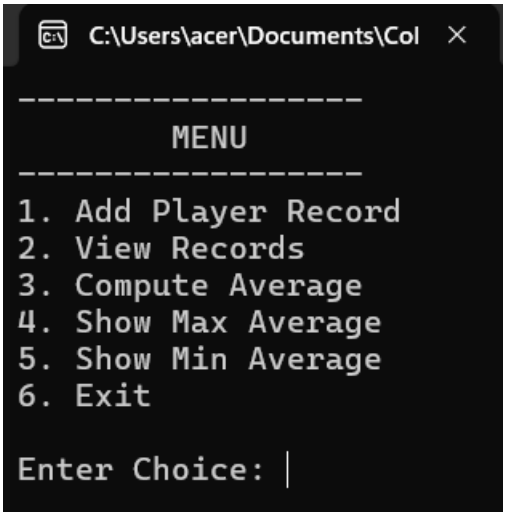
1. Inside the main function:
   - Display a menu with options for adding a record, viewing records, computing average, and more.
   - Use a loop to repeatedly prompt the user for their choice and perform the corresponding action.
   - Based on the user's choice, call the appropriate functions.

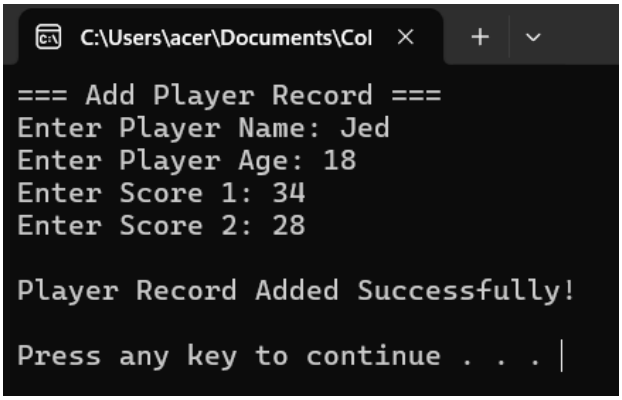
Step 5: Testing and Further Improvements

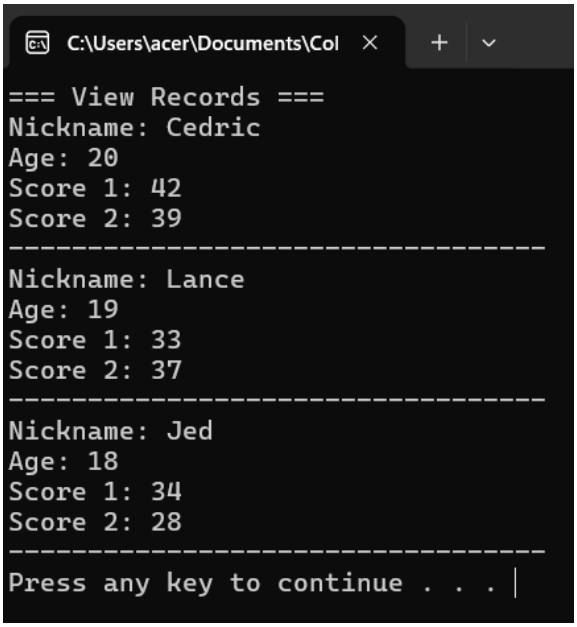
1. Compile and run your program.
2. Test each menu option to ensure that the program functions correctly.
3. Consider implementing the showMaxAverage and showMinAverage functions by iterating through the linked list to find players with the highest and lowest averages.

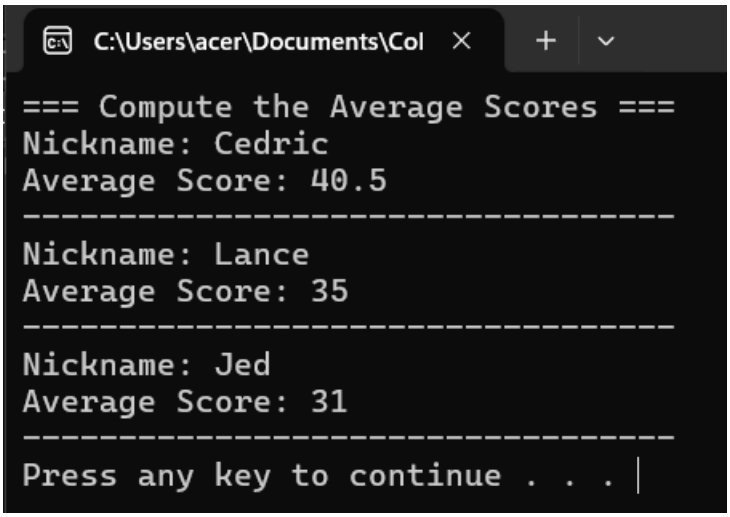
Step 6: Submission

Submit your completed <lastname>_E6_linked_list.cpp file and include an output screenshot here:





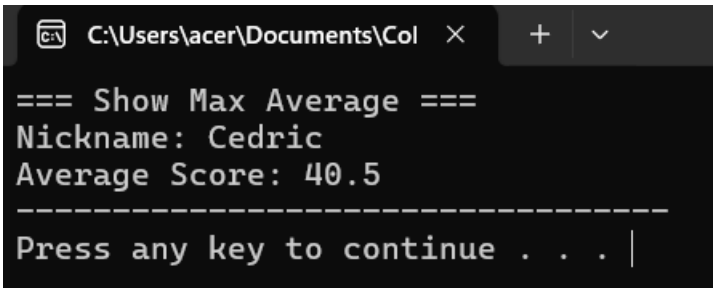



C:\Users\acer\Documents\Col
=== Compute the Average Scores ===
Nickname: Cedric
Average Score: 40.5

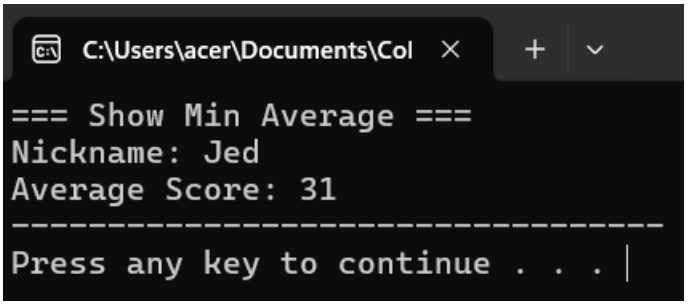
Nickname: Lance
Average Score: 35

Nickname: Jed
Average Score: 31

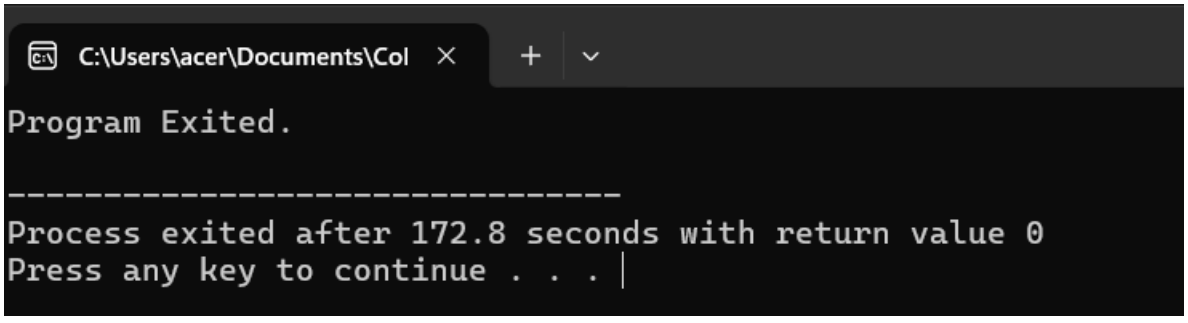
Press any key to continue . . .


C:\Users\acer\Documents\Col
=== Show Max Average ===
Nickname: Cedric
Average Score: 40.5

Press any key to continue . . .


C:\Users\acer\Documents\Col
=== Show Min Average ===
Nickname: Jed
Average Score: 31

Press any key to continue . . .


C:\Users\acer\Documents\Col
Program Exited.

Process exited after 172.8 seconds with return value 0
Press any key to continue . . .

VII. QUESTION AND ANSWER

Briefly answer the questions below. Avoid erasures. For group activity, specify the name of GROUP MEMBER/s who answered the question. Do not forget to include the source for all NON-ORIGINAL IDEAS.

• What are the advantages of using linked lists? Linked lists provide benefits such as dynamic memory allocation, streamlined insertion and deletion processes, and adaptable sizing. They don't rely on contiguous memory allocation like arrays do, enabling more efficient memory utilization and simpler implementation of structures like stacks and queues.
• What are the disadvantages of linked lists? The disadvantages of linked lists include higher memory usage because of the need for extra pointers per element, slower access times compared to arrays for non-sequential access, and potential challenges in efficiently traversing the list without sequential access. Moreover, linked lists may not optimize memory cache as effectively as arrays due to their non-contiguous memory arrangement.

VIII. REFERENCES

- Zak, Dianne (2016). An Introduction to Programming with C++
- Deitel, Paul & Deitel, Harvey (2012). C++ How To Program, Eighth Edition
- https://www.cprogramming.com/tutorial/lesson15.html